



CHALLENGE 10 / 20

THE CLOUD WITNESS THAT WASN'T SYNCED

FORENSICS OF EPHEMERAL CLOUD INFRASTRUCTURE

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A fintech platform with 120 million users runs 900 microservices across two cloud providers and an edge network. Containers live for an average of eleven minutes and serverless functions for seconds. A breach was found 23 days after it began, from an outside tip. By then containers had been destroyed more than 200,000 times.

Log retention ranges from seven to thirty days. Some pipelines are sampled. Each provider's audit trail has a different level of detail. Clocks differ by 80 milliseconds to six minutes, and one managed service delays its logs by up to forty minutes. The attacker used stolen workload identity tokens, created short-lived functions and touched storage and a secrets manager.

The early days sit outside nearly all retention. Some evidence survives only as billing records, quota metrics and flow aggregates.

2 | PROBLEM STATEMENT

Develop an **Ephemeral Infrastructure Reconstruction System** that reconstructs attacker activity on compute resources that no longer exist, using heterogeneous, expired, sampled and clock-skewed fragments, including telemetry never meant for security.

The system must map where evidence exists and where it does not, and compute the maximum reach the attacker's identities could have had inside every blind spot.

Hard constraints

- The earliest part of the breach is beyond the retention of most sources.
- Sampling, delay and clock skew differ per service and per provider.
- Billing, quota and flow data must be used as evidence.
- Identities and resources must be mapped across providers.
- The system must handle at least a hundred thousand resource lifecycles.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Resource Lifecycle Reconstruction	Ability to rebuild lifecycles of resources that no longer exist.
02	Clock and Delay Normalisation	Ability to align sources with different skew and delay.
03	Non-Security Telemetry Evidence	Ability to use billing, quota and flow data as evidence.
04	Cross-Provider Identity Mapping	Ability to follow one identity across providers.
05	Evidence Coverage Mapping	Ability to show what is covered, partial and blind.
06	Blind-Spot Maximum Reach	Ability to bound what the attacker could have reached in a blind spot.
07	Sampling-Aware Inference	Ability to reason correctly from sampled sources.
08	Scale Handling	Ability to process very large numbers of short-lived resources.

The system must clearly distinguish between:

COVERED	PARTIALLY COVERED	BLIND SPOT	INFERRED	UNKNOWN
---------	-------------------	------------	----------	---------

An activity window that has no supporting evidence must not be presented as clean.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated two-provider ephemeral environment
02	Destroyed resources, expired logs and skewed clocks
03	Reconstruction of attacker activity on destroyed resources
04	Use of billing, quota and flow data
05	Coverage map across the breach period
06	Maximum-reach analysis for each blind spot
07	Performance at the required scale

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.